

Arquitecturas Web

Arquitectura cliente-servidor

El modelo de desarrollo web se apoya, en una primera aproximación desde un punto de vista centrado en el hardware, en lo que se conoce como arquitectura cliente-servidor ¹⁾ que define un patrón de arquitectura donde existen dos actores, cliente y servidor, de forma que el primero es quién se conecta con el segundo para solicitar algún servicio. En el caso que nos ocupa, el desarrollo web, los clientes solicitan que se les sirva una web para visualizarla, aunque también es posible solicitar información si hablamos del caso de los servicios web que también veremos más adelante. En cualquier caso, en ambos casos aparece el mismo escenario, donde un servidor se encuentra ejecutándose ininterrumpidamente a la espera de que los diferentes clientes realicen una solicitud.

Normalmente a la solicitud que hacen los clientes al servidor se le llama petición (request) y a lo que el servidor devuelve a dicho cliente le llamamos respuesta (response).

También hay que tener en cuenta que esta arquitectura cliente-servidor plantea la posibilidad de numerosos clientes atendidos por un mismo servidor. Es decir, el servidor será un software multitarea que será capaz de atender peticiones simultáneas de numerosos clientes.

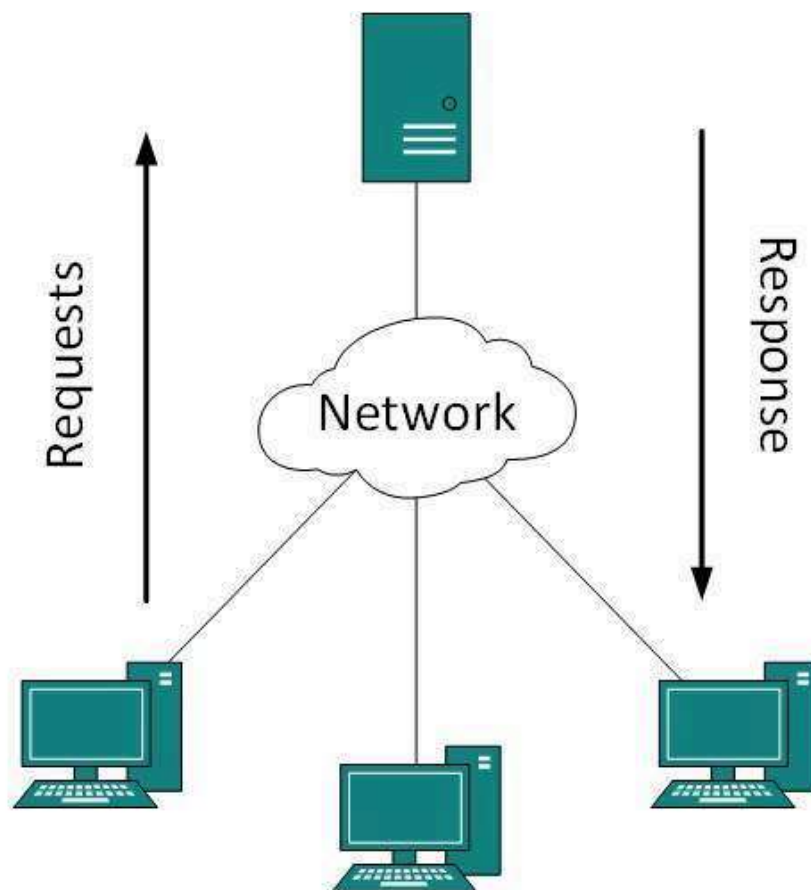


Figure 1: Arquitectura cliente-servidor

Desde un punto de vista de desarrollo una aproximación más detallada para este modelo de ejecución es lo que se conoce como modelo en 3 capas ²⁾. Es un modelo donde se muestra más en detalle como se distribuye el software que participa en cualquier desarrollo web. Sigue estando presente la arquitectura cliente-servidor (todo se basa en ella) pero aparecen más detalles como el software utilizado en cada uno de los dos actores y como interactúan las diferentes tecnologías o aplicaciones.

Protocolos

La pila de protocolos en los que se basa Internet es muy amplia, ya sea siguiendo el modelo OSI o el modelo TCP. En cualquier caso, en el tema que nos ocupa a nosotros sólo nos fijaremos en la última capa, la capa de transporte. Es en esta capa donde están los protocolos de la web, los que usan navegadores y servidores (web y aplicaciones) para comunicarse. Al fin y al cabo, la web no es más que una de las tantas aplicaciones que existen en Internet. Otras aplicaciones, que también veremos en este curso, son FTP y SSH, entre otras.

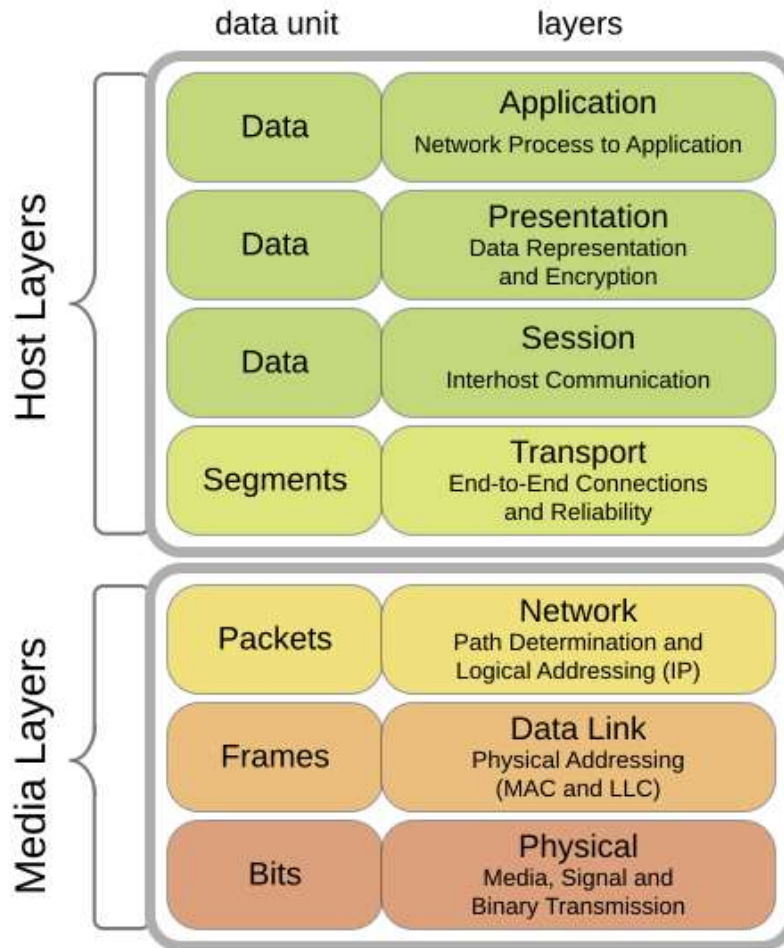


Figure 2: Modelo OSI (Fuente: Wikipedia)

Puesto que en esta asignatura nos centramos principalmente en la web y, aunque en menor medida, en los protocolos (de aplicación) que de alguna manera son de utilidad para trabajar con ella, nos centraremos exclusivamente en ellos:

- **HTTP:** HyperText Transfer Protocol. Protocolo de comunicación para la web
- **HTTPS:** HTTP Secure. Protocolo seguro de comunicación para la web. Surge de aplicar una capa de seguridad, utilizando SSL/TLS, al protocolo HTTP
- **Telnet:** Es un protocolo que establece una línea de comunicación basada en texto entre un cliente y un servidor. Desde su aparición se utilizó ampliamente como vía de comunicación remota con el sistema operativo ya que permitía la ejecución remota de comandos. Con el tiempo ha ido cayendo en desuso a favor de un protocolo seguro que lo sustituye, SSH.
- **SSH:** Secure Shell. Protocolo seguro de comunicación ampliamente utilizado para la gestión remota de sistemas, ya que permite la ejecución remota de comandos. Surge como reemplazo para el protocolo no seguro Telnet.
- **SCP:** Secure Copy. Es un protocolo seguro (basado en RCP, Remote Copy) que permite transferir ficheros entre un equipo local y otro remoto o entre dos equipos remotos. Utiliza SSH por lo que garantiza la seguridad de la transferencia así como de la autenticación de los usuarios.
- **FTP:** File Transfer Protocol. Es un protocolo que se utiliza para la transferencia de archivos entre un equipo local y otro remoto. Su principal problema es que tanto la autenticación como la transferencia se realiza como texto plano, por lo que se considera un protocolo no seguro.
- **SFTP:** SSH FTP. Es una versión del protocolo FTP que utiliza SSH para cifrar tanto la autenticación del usuario como la transferencia de los archivos. Es la opción segura al uso de un protocolo como FTP

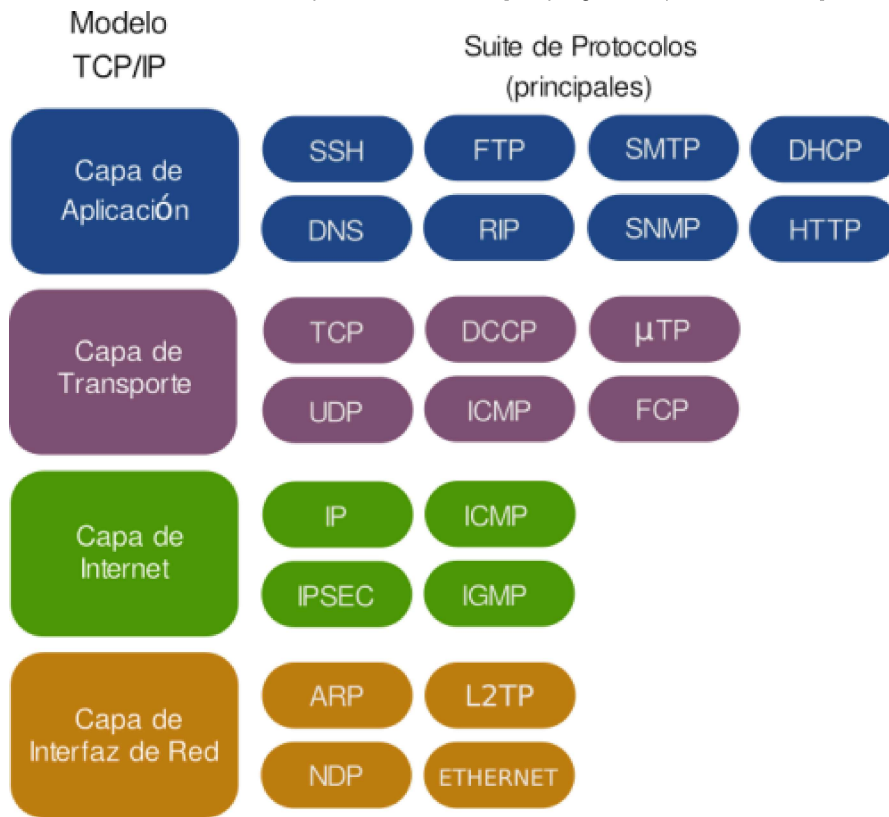


Figure 3: Protocolos en Internet (Fuente:Wikipedia)

Protocolo HTTP

El protocolo HTTP es un protocolo para la transferencia de páginas web (hipertexto) entre los clientes (navegadores web) y un servidor web. Cuando un usuario, a través del navegador, quiere un documento (página web), éste lo solicita mediante una petición HTTP al servidor. Éste le contestará con una respuesta HTTP y el documento, si dispone de él.

Hay que tener en cuenta que, al contrario que el resto de protocolos que estamos viendo en esta parte, HTTP no tiene estado. Eso significa que un servidor web no almacena ninguna información sobre los clientes que se conectan a él. Así, cada petición/respuesta supone una conexión única y aislada. En cualquier caso, utilizando tecnologías en el lado servidor es posible escribir aplicaciones web que puedan establecer sesiones o cookies para almacenar ese estado y “recordar” de alguna manera a los clientes en sucesivas conexiones.

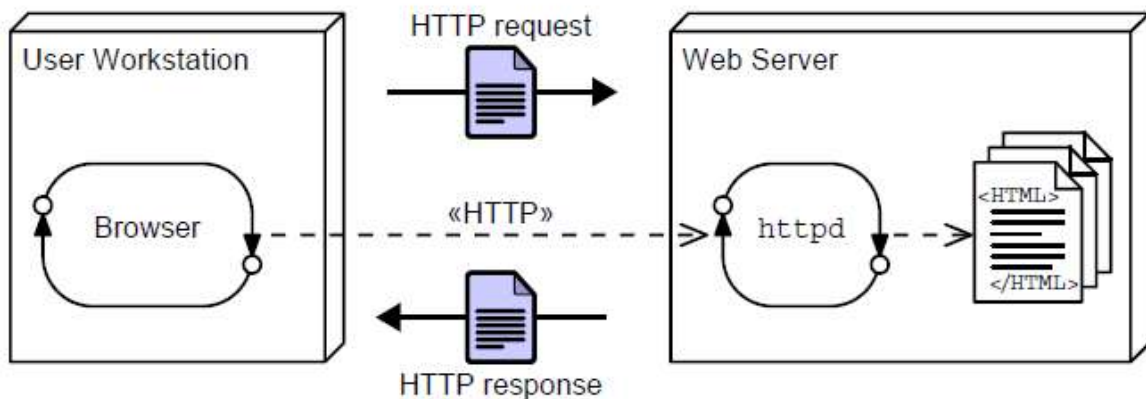


Figure 4: Protocolo HTTP

A continuación, a modo de ejemplo, podemos ver una petición HTTP que un navegador (*Firefox*) ha realizado a un sitio web (*misitio.com*), solicitando el documento *index.html*.

```
GET /index.html HTTP/1.1
Host: www.misitio.com
User-Agent: cliente
Referer: www.google.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:45.0) Gecko/20100101 Firefox/45.0
Connection: keep-alive
[Línea en blanco]
```

Y el servidor web le contesta con el contenido del documento para que el navegador que lo ha solicitado lo pueda renderizar para que el usuario lo visualice en su pantalla:

```
HTTP/1.1 200 OK
Date: Fri, 31 Dec 2003 23:59:59 GMT
Content-Type: text/html
Content-Length: 1221
```

```
<html lang="es">
<head>
<meta charset="utf-8">
<title>Mi título</title>
</head>
<body>
<h1>Bienvenido a mi sitio.com</h1>
. . .
. . .
</body>
</html>
```

Protocolo SSL/TLS

SSL (Secure Sockets Layer) y TLS (Transport Layer Security) son protocolos de cifrado que se utilizan para cifrar las comunicaciones en Internet. En ocasiones se hace referencia en ambos casos al uso de SSL pero la realidad es que TLS es el sucesor de SSL debido a las diferentes vulnerabilidades que han ido surgiendo de este último.

En este caso, la aplicación de esta capa de seguridad, cifrando las comunicaciones del protocolo HTTP, da lugar a lo que se conoce como HTTPS, que veremos a continuación.

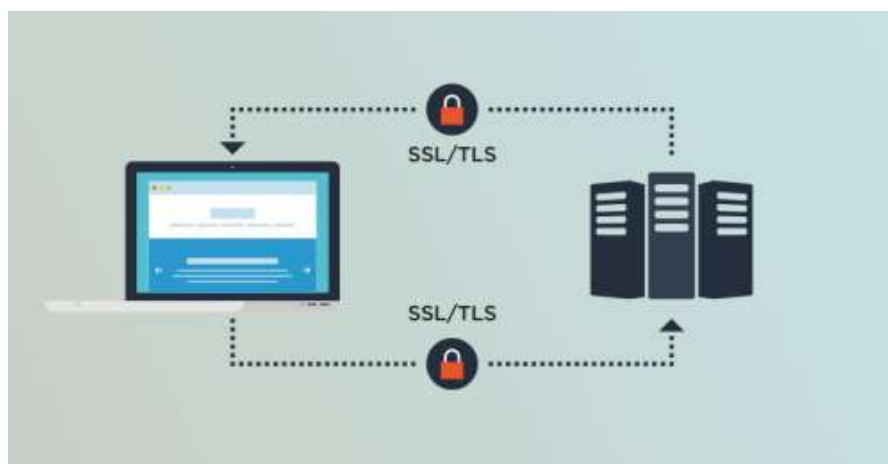


Figure 5: Protocolo SSL/TLS

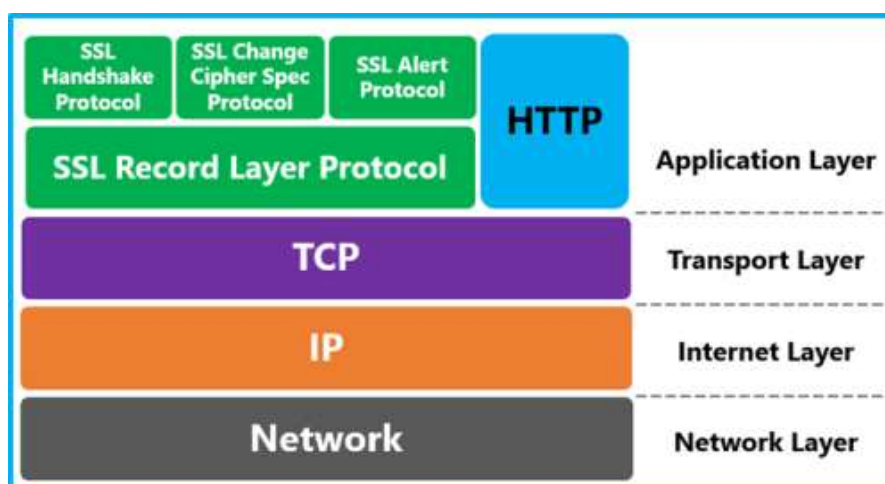


Figure 6: SSL/TLS en el modelo TCP

Protocolo HTTPS

El protocolo HTTPS (HTTP Secure) es un protocolo de comunicación segura a través de Internet. Este protocolo se basa en la comunicación del protocolo HTTP pero con una capa de seguridad adicional cifrando el contenido con TLS ó SSL.

Su principal utilidad es el cifrado de los mecanismos de autenticación en la web, justamente cuando el usuario envía sus credenciales al servidor para validar su sesión. Es el momento más crítico en una comunicación, aunque actualmente se está utilizando abiertamente durante toda la comunicación entre cliente y servidor en la web por privacidad e integridad.

Con respecto a la integridad, HTTPS proporciona un mecanismo de autenticación con respecto al sitio web y servidor web que estamos visitando, evitando así ataques como el del *Man in the Middle*. Es la manera en la que podemos estar seguros de que el sitio con el que estamos comunicándonos es el que creemos que es.

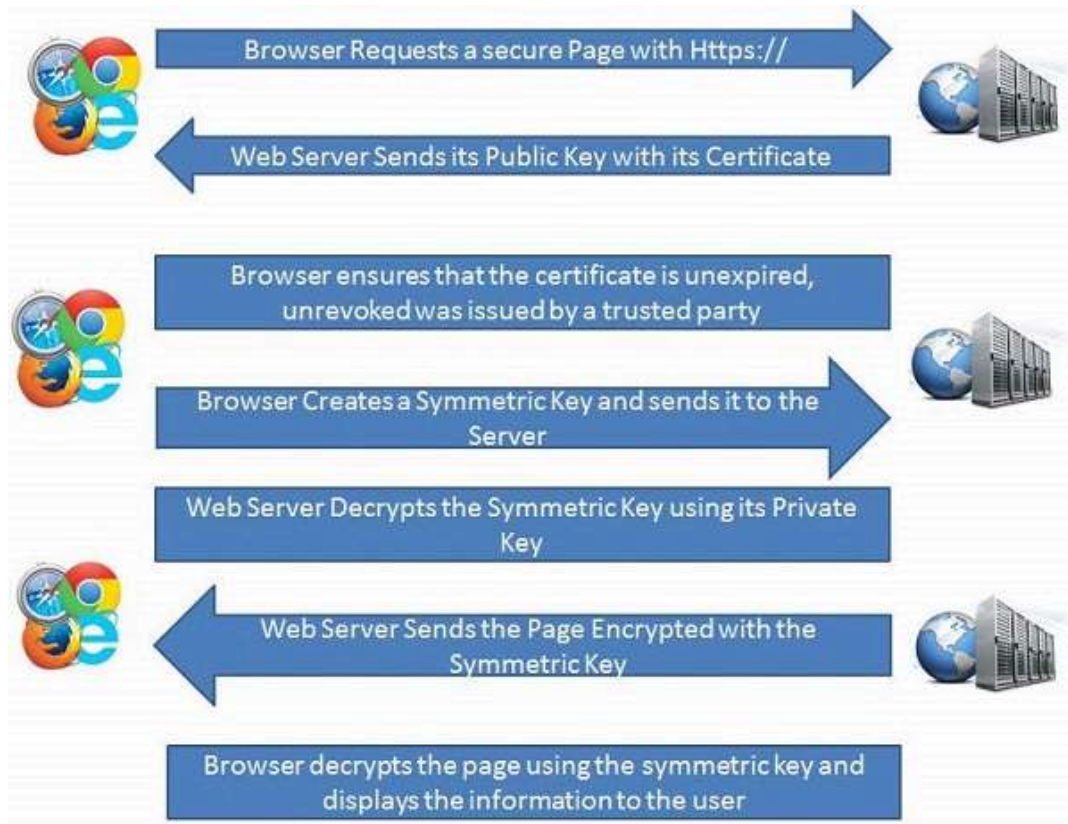


Figure 7: Protocolo HTTPS

Protocolo Telnet

El protocolo Telnet (**Te**letype **net**work) surgió como una forma de proporcionar una comunicación interactiva basada en texto entre dos máquinas. Telnet proporciona acceso remoto a otra máquina y permite ejecutar comandos en ella y visualizar los resultados en pantalla. Para ello, en la máquina remota debía estar ejecutándose el lado servidor y, en el lado cliente, el usuario debía utilizar una aplicación cliente para establecer la conexión y más adelante los comandos como si de la línea de comandos del sistema operativo se tratara.

Con el tiempo, y debido a la falta de seguridad en las comunicaciones y la autenticación de los usuarios, se abandonó el uso de este protocolo en beneficio de otros como SSH.

Protocolo SSH

El protocolo SSH (**Secure SHell**), al igual que Telnet, proporciona una forma de comunicación entre dos máquinas remotas, basada en texto pero en este caso tanto la autenticación como la propia comunicación se encuentran cifradas. Su uso más habitual está en la gestión remota de máquinas Unix/Linux, tal y como ocurría en su momento con Telnet.

Además, otros protocolos como FTP (por eso llamado SFTP) o SCP se apoyan en este protocolo de cifrado para cifrar sus comunicaciones.

Protocolo SCP

El protocolo SCP (**Secure CoPy**), basado en el protocolo SSH, permite copiar ficheros entre dos equipos, tanto del equipo local al remoto como en el sentido contrario.

Protocolo FTP

El protocolo FTP (**File Transfer Protocol**) es un protocolo para la transferencia de ficheros entre dos máquinas: una máquina cliente y otra máquina remota o servidor donde se alojan dichos ficheros. La idea es que la máquina remota sirva como repositorio de información y sean los múltiples clientes los que se conectan a ella para coger o subir ficheros.

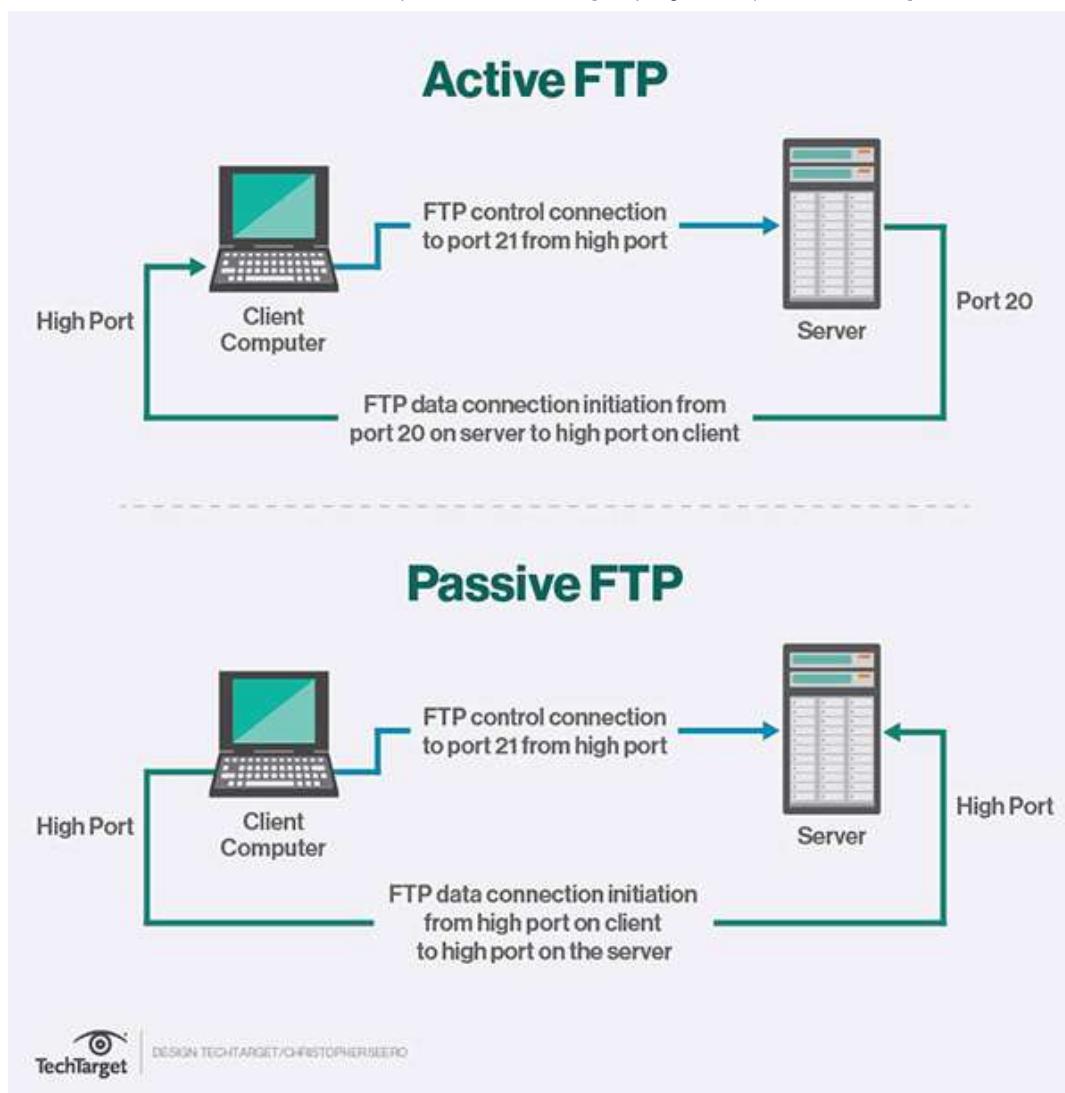


Figure 8: Protocolo FTP (activo y pasivo)

Protocolo SFTP

El protocolo SFTP (SSH FTP) es la versión segura del protocolo FTP. Esta vez, a diferencia de lo que ocurre con el protocolo FTP, las comunicaciones se cifran de la misma forma que en el protocolo SSH.

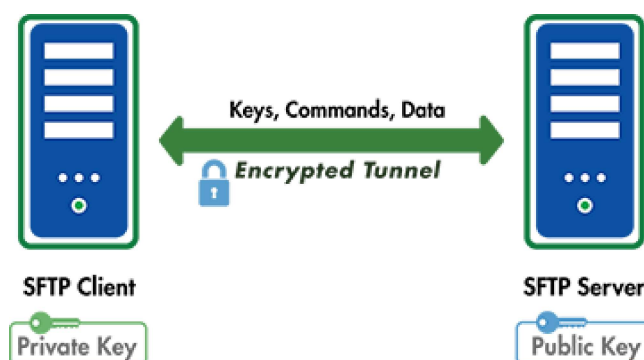


Figure 9: Protocolo SFTP

Servidores web y de aplicaciones

La realidad es que existen una infinidad de servidores web³⁾ y de aplicaciones⁴⁾. Aunque a la hora de la verdad, unos pocos de ellos (4 ó 5) se reparten más del 90% de los sitios web en Internet. Los demás, soluciones quizás muy específicas, no es que no sean buenas aplicaciones sino que se utilizan en ámbitos muy concretos. Los más utilizados tienen un perfil más generalista y la gran mayoría de desarrolladores y administradores se decantan por ellos porque cumplen con los requisitos que se necesitan.

Estructura de un servidor web

Instalación y funcionamiento de servidores web

Instalación en Windows

Para la instalación del servidor web Apache en Windows tenemos que hacernos con algunos de los empaquetados con XAMPP [https://www.apachefriends.org/index.html] debido a que el proyecto Apache ha decidido dejar de preparar instaladores para este sistema⁵.

En nuestro caso contamos con el instalador XAMPP, que además añade paquetes opcionales muy necesarios como:

- MySQL
- PHP
- Filezilla (servidor FTP)
- Servidor de correo
- Apache Tomcat
- phpMyAdmin

Además, incorpora un panel de control desde donde podemos iniciar/detener los diferentes servidores instalados en el caso de que hayamos decidido no instalarlos como servicios del Sistema Operativo. En cualquier caso, la opción recomendada es la de hacerlo como servicios puesto que de esa manera estos servicios arrancan automáticamente en el arranque del sistema y se detienen automáticamente cuando este se apaga. Es la mejor manera de garantizar la disponibilidad total puesto que no tenemos que estar pendientes de ellos.

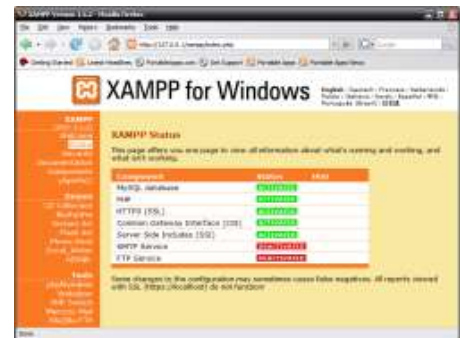
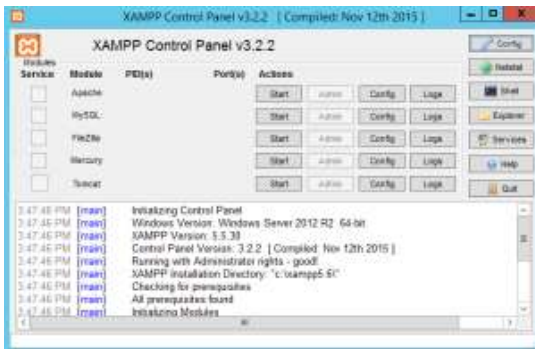


Figure 10: Solución XAMPP

Instalación en Linux

Para la instalación en Linux, puesto que usaremos la distribución Debian, lo haremos utilizando la herramienta apt que nos permitirá instalar el servidor web y todas sus dependencias sin mayor problema.

```
santi@zenbook:~$ sudo apt-get install apache2 apache2-utils
```

Una vez instalado, podremos encontrarnos con el directorio etc/apache2 donde se almacenan los ficheros de configuración y /var/www/html que es la carpeta que Apache tiene configurada por defecto para almacenar las páginas web del sitio principal.

Para comprobar que hasta el momento todo funciona, podemos visitar la web que por defecto se instala visitándola desde nuestro equipo. Para eso, introducimos la IP en el navegador de la máquina donde hemos instalado Apache y tendremos que ver la web que por defecto se instala:

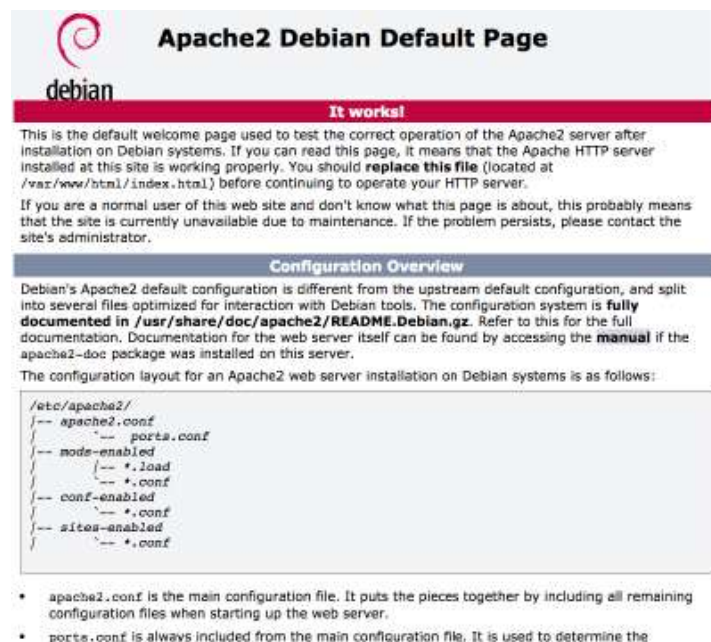


Figure 11: Página por defecto de Apache

Además, instalaremos todas las herramientas que ya instalamos en Windows a través de *XAMPP*. En este caso lo haremos instalándolas una a una con apt.

Debian - Instalación de Apache



Instalación de MySQL

El siguiente comando instalará la versión disponible de *MySQL* en la versión 9 de Debian que es la que usamos. Hay que tener en cuenta que realmente se instalará *MariaDB* (versión 10.1), que es el fork libre que surgió de *MySQL* y por el que ha optado Debian desde entonces. Es totalmente compatible con *MySQL* hasta el momento, por lo que no tendremos ningún problema a la hora de trabajar con ambos.

```
santi@zenbook:$ sudo apt-get install mysql-server
```

Instalación de PHP

También tenemos que instalar el intérprete para lenguaje *PHP* y el módulo para que *Apache* pueda trabajar con él:

```
santi@zenbook:$ sudo apt-get install php libapache2-mod-php7.0
```

Instalación de phpMyAdmin

Ahora instalamos la herramienta *phpMyAdmin* para administrar nuestro servidor de Bases de Datos:

```
santi@zenbook:$ sudo apt-get install phpmyadmin
```

Debian - Instalar PHP, MySQL y phpMyAd...



Instalación de un servidor FTP

También instalaremos un servidor de FTP para que sea posible desplegar los sitios web que se desarrollen. En nuestro caso usaremos un servidor FTP seguro como *Pure-FTPd*

```
santi@zenbook:$ sudo apt-get install pure-ftpd
```

La instalación por defecto iniciará el servicio en el puerto 21 y podremos utilizar nuestra cuenta de usuario del sistema como cuenta para acceder por FTP. Simplemente tendremos que tener en cuenta que por defecto accederemos a la carpeta *home* y a nosotros nos interesa acceder a la carpeta donde *Apache* espera encontrar las webs que debe servir a los visitantes. Pero incluso aunque nos posicionemos “manualmente” (o configurando la configuración de nuestro despliegue desde el IDE que usemos) nos encontraremos con que no tenemos permisos sobre dicha carpeta.

Si queremos poder escribir sobre la ruta `/var/www/html` deberemos configurar los permisos correctos o reasignar usuarios y grupos a dicha carpeta. Podemos, por ejemplo, crear un grupo de desarrolladores, incluir nuestro usuario a dicho grupo y permitir que éste pueda escribir en la carpeta:

```
santi@zenbook:$ sudo addgroup dev
Adding group `dev` (GID 1001) ...
Done
santi@zenbook:$ sudo adduser santi dev
Adding user `santi` to group `dev` ...
Adding user santi to group dev
Done
santi@zenbook:$ cd /var/www
santi@zenbook:$ sudo chown root.dev html
santi@zenbook:$ sudo chmod g+w html
```

Ahora nuestro usuario (santi en este caso) ya podrá copiar archivos a la carpeta que utilizará *Apache* por defecto.



Estructura de un servidor de aplicaciones

Instalación y funcionamiento de servidores de aplicaciones

Instalación en Windows

Puesto que utilizamos *XAMPP* en Windows y *Apache Tomcat* viene como opción, siguiendo las instrucciones de la instalación de XAMPP de la parte de instalación de servidores web podéis ver como hacerlo. Una vez instalado la configuración es común tanto para Windows y Linux por lo que se puede acceder directamente a ese apartado.

Instalación en Linux

El servidor de aplicaciones *Apache Tomcat* [<http://tomcat.apache.org>] no dispone de instalador por lo que la manera habitual de instalarlo es descargar la versión que queramos (actualmente la última versión es la 9) y descomprimirlo en la carpeta que queramos.

```
santi@zenbook:$ tar xvfz apache-tomcat-9.0.1.tar.gz
x apache-tomcat-9.0.1/conf/
x apache-tomcat-9.0.1/conf/catalina.policy
x apache-tomcat-9.0.1/conf/catalina.properties
x apache-tomcat-9.0.1/conf/context.xml
x apache-tomcat-9.0.1/conf/jaspic-providers.xml
x apache-tomcat-9.0.1/conf/jaspic-providers.xsd
x apache-tomcat-9.0.1/conf/logging.properties
x apache-tomcat-9.0.1/conf/server.xml
x apache-tomcat-9.0.1/conf/tomcat-users.xml
x apache-tomcat-9.0.1/conf/tomcat-users.xsd
x apache-tomcat-9.0.1/conf/web.xml
x apache-tomcat-9.0.1/bin/
. . .
. . .
```

Para iniciarlo, dispone de un script `startup.sh` que lo lanzará en segundo plano:

```
santi@zenbook:$ ./startup.sh
Using CATALINA_BASE:   /Users/Santi/apache-tomcat-7.0.67
Using CATALINA_HOME:   /Users/Santi/apache-tomcat-7.0.67
Using CATALINA_TMPDIR: /Users/Santi/apache-tomcat-7.0.67/temp
Using JRE_HOME:        /Library/Java/JavaVirtualMachines/jdk1.8.0_25.jdk/Contents/Home
```

```
Using CLASSPATH: /Users/Santi/apache-tomcat-7.0.67/bin/bootstrap.jar:/Users/Santi/apache-tomcat-7.0.67/bin/tomcat-juli.jar
Tomcat started.
```

Y también de un script shutdown.sh que lo detiene:

```
santi@zenbook:~$ ./shutdown.sh
Using CATALINA_BASE: /Users/Santi/apache-tomcat-7.0.67
Using CATALINA_HOME: /Users/Santi/apache-tomcat-7.0.67
Using CATALINA_TMPDIR: /Users/Santi/apache-tomcat-7.0.67/temp
Using JRE_HOME: /Library/Java/JavaVirtualMachines/jdk1.8.0_25.jdk/Contents/Home
Using CLASSPATH: /Users/Santi/apache-tomcat-7.0.67/bin/bootstrap.jar:/Users/Santi/apache-tomcat-7.0.67/bin/tomcat-juli.jar
```

Instalación en Debian utilizando apt

Si trabajamos con Debian existe la posibilidad de instalar Tomcat utilizando la herramienta *apt*. En el ejemplo siguiente instalamos el servidor de aplicaciones y todas herramientas y documentación (vienen como paquetes a parte en Debian):

```
santi@zenbook:~$ sudo apt-get install tomcat8 tomcat8-*
```

En este caso, si queremos iniciar/detener/reiniciar el servidor, puesto que éste se encontrará ahora instalado como un servicio en nuestro sistema Linux tendremos que hacerlo como habitualmente se hace:

```
santi@zenbook:~$ sudo service tomcat8 start
santi@zenbook:~$ sudo service tomcat8 restart
santi@zenbook:~$ sudo service tomcat8 stop
```

También tenemos que tener en cuenta que, en este caso, todos los ficheros de configuración de *Apache Tomcat* se encuentran en `/etc/tomcat8`.

Configuración básica

En cualquier caso, al tratarse de un servidor de aplicaciones, lo habitual es que *escuche* en el puerto 8080, aunque hay que tener en cuenta que se puede cambiar en el caso de que en el equipo donde se encuentra instalado no haya un servidor web y además queramos que nuestra aplicación web quede expuesta al exterior. Nos interesará, por tanto, modificar el puerto al 80 para mayor comodidad a la hora de acceder a nuestra aplicación desde el navegador.

Para cambiar el puerto donde *Apache Tomcat* escuchará tenemos que editar el fichero `conf/server.xml` y cambiar el número del conector del protocolo HTTP:

```
...
<Connector port="8080" protocol="HTTP/1.1"
    connectionTimeout="20000"
    redirectPort="8443" />
...
```

Independientemente del puerto que hayamos configurado, si accedemos a la página principal del servidor de aplicaciones nos encontraremos la portada:

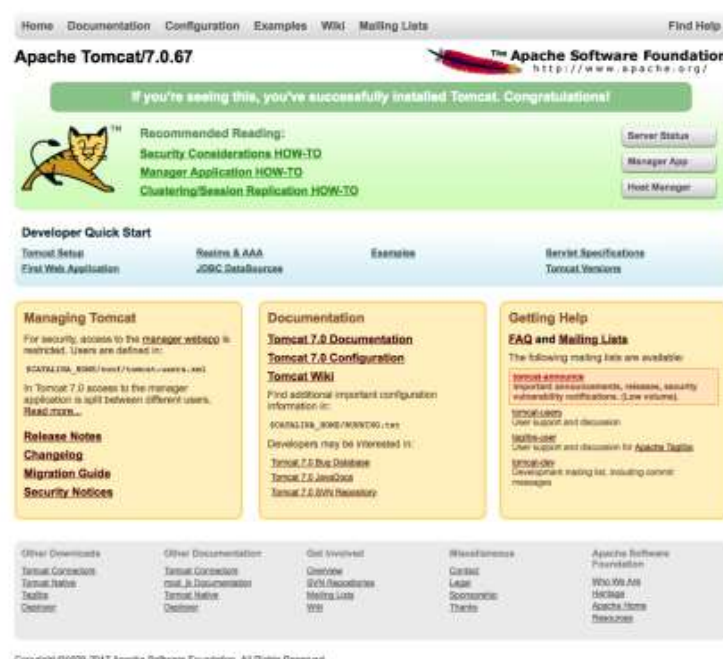


Figure 12: Home Apache Tomcat

Desde donde podemos acceder a diferentes secciones donde podemos monitorizar/configurar algunos aspectos del servidor

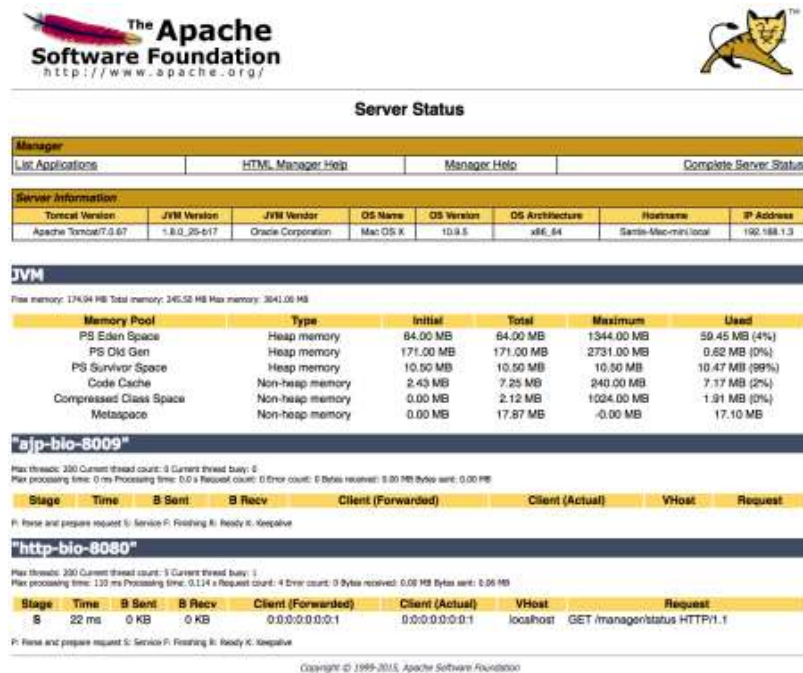


Figure 13: Estado del servidor

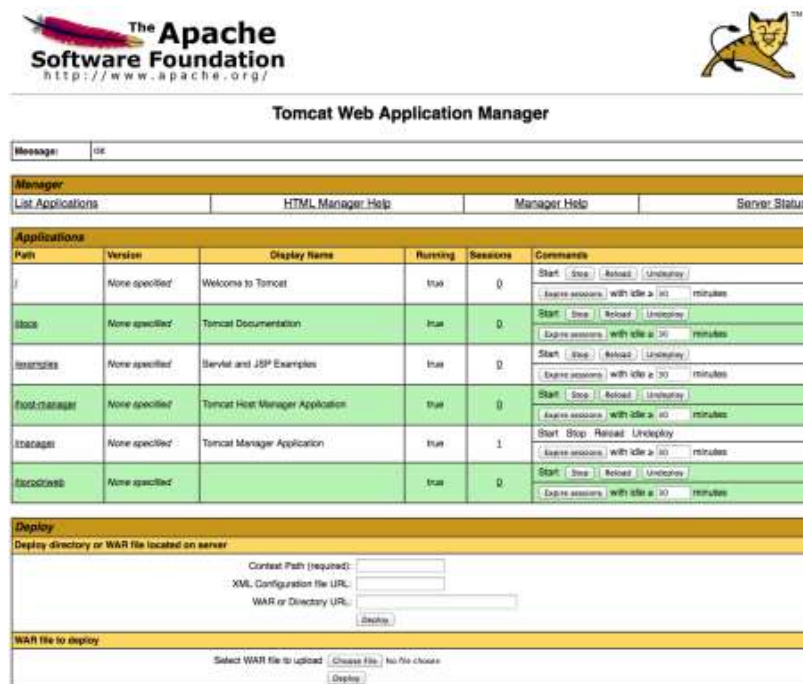


Figure 14: Listado de aplicaciones web desplegadas

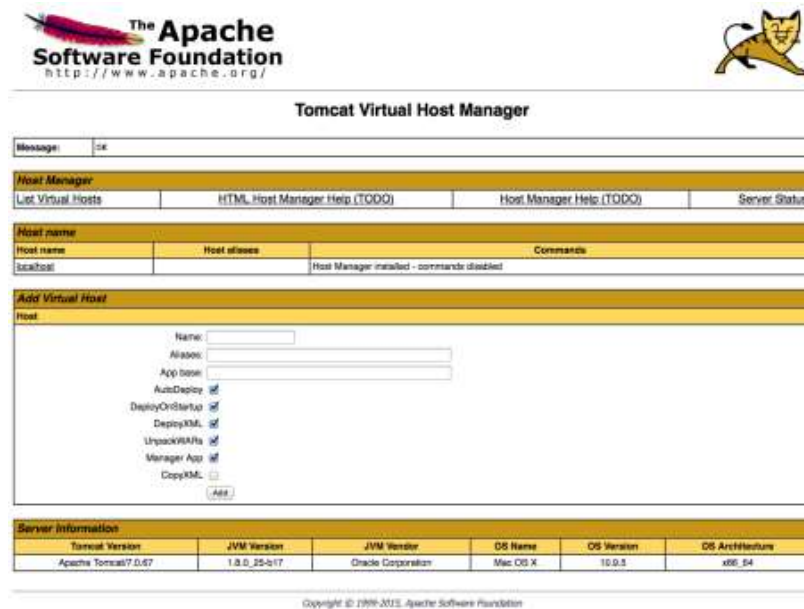


Figure 15: Listado de hosts virtuales configurados

Pero antes de poder acceder a las diferentes secciones del servidor desde el navegador hay que tener en cuenta que, por seguridad, dicho acceso se encuentra restringido y tendremos que registrar usuarios en `conf/tomcat-users.xml` con nombre de usuario, contraseña y los roles con los que accederán.

Podemos crear un usuario para cada rol o bien un usuario que tenga varios de ellos (incluso todos). Cada rol definido por *Tomcat* da acceso a una de las partes del servidor:

- **manager-status:** Da acceso a la sección donde monitorizar el estado de Tomcat
- **manager-gui:** Da acceso al listado de aplicaciones y a poder desplegarlas desde la web
- **admin-gui:** Da acceso a la parte de administración de hosts virtuales

La forma en como podemos editar el fichero y crear nuevos usuarios puede verse en el siguiente fragmento del fichero configuración extraído de una instalación limpia de *Tomcat* y modificado con un usuario `tomcat` y contraseña `tomcat` con todos los roles.

conf/tomcat-users.xml

```
<tomcat-users>
<!--
NOTE: By default, no user is included in the "manager-gui" role required
to operate the "/manager/html" web application. If you wish to use this app,
you must define such a user - the username and password are arbitrary.
-->
<!--
NOTE: The sample user and role entries below are wrapped in a comment
and thus are ignored when reading this file. Do not forget to remove
<!-- ... --> that surrounds them.
-->

<role rolename="tomcat"/>
<user username="tomcat" password="tomcat" roles="manager-status, manager-gui, admin-gui"/>

</tomcat-users>
```

Ejercicios

1. Crea un grupo de desarrolladores con permisos para escribir en el DocumentRoot de *Apache* y agrega dos usuarios a dicho grupo. Comprueba que ambos pueden subir contenido a la carpeta a través de un servidor FTP
2. Modifica la contraseña para acceder al panel de administración de Tomcat (Windows y Linux)
3. Instala Tomcat 9 descargándolo de la web en Windows y Linux. Comprueba que funciona correctamente en ambos Sistemas Operativos



Proyectos de Ejemplo

Prácticas

© 2017 Santiago Faci

1)

https://en.wikipedia.org/wiki/Client-server_model

2)

https://en.wikipedia.org/wiki/Multitier_architecture

3)

https://en.wikipedia.org/wiki/Comparison_of_web_server_software

4)

https://en.wikipedia.org/wiki/List_of_application_servers

5)

<http://httpd.apache.org/docs/current/platform/windows.html#down>

apuntes/introduccion.txt · Last modified: 2019/01/04 13:02 by 127.0.0.1